

VICTORIOSA

Prompts maestros para crear una tienda tipo Shopify + dropshipping + IA comercial

Repositorio y Supabase ya incorporados

Repo: <https://github.com/FileLanderScanner/Victoriosa-marketplace.git>

Supabase URL: <https://ngliugfcwydnfbpalkpb.supabase.co>

Uso recomendado

- Pegá los prompts de a uno en Copilot Agent o Codex, en el orden del documento.
- No pegues claves secretas dentro del chat. Las claves van en .env.local, Vercel o Supabase.
- Los proveedores externos deben conectarse por APIs autorizadas, CSV o carga manual. Evitar scraping no autorizado.
- El primer MVP debe vender por WhatsApp y consulta; pagos reales se activan en una fase posterior.

Indice operativo

- Datos que ya tenemos y datos que faltan
- Prompt 00 - Director maestro de Victoriosa
- Prompt 01 - Preflight, repo y rama segura
- Prompt 02 - Bootstrap Next.js + arquitectura base
- Prompt 03 - Supabase foundation segura
- Prompt 04 - Modelo ecommerce y migraciones
- Prompt 05 - Storefront publico
- Prompt 06 - Admin tipo Shopify simplificado
- Prompt 07 - Importador CSV/JSON y proveedores mock
- Prompt 08 - Pricing, margen y scoring rentable
- Prompt 09 - AI Commerce Agent
- Prompt 10 - WhatsApp, leads y carrito de consulta
- Prompt 11 - Seguridad, RLS y QA
- Prompt 12 - Deploy preview Vercel y variables
- Prompt 13 - Growth, SEO y venta piloto
- Prompt 14 - Reporte final y continuidad autonoma
- Prompt extra - Mensaje corto para Copilot

Datos que ya tenemos

Dato	Valor
Repositorio GitHub	https://github.com/FileLanderScanner/Victoriosa-marketplace.git
Supabase URL	https://ngliugfcwydnfbpalkpb.supabase.co
Supabase project ref inferido	ngliugfcwydnfbpalkpb
Marca	Victoriosa
Modelo objetivo	Tienda propia tipo Shopify simplificado + dropshipping preparado + agente IA comercial
Pais/moneda inicial	Uruguay / UYU

Datos que conviene brindar para que el prompt quede 100% operativo

Area	Dato a completar
GitHub	Confirmar que VS Code/Copilot/Codex tiene acceso al repo y permiso para crear ramas, commits y PRs.
Supabase anon key	Clave anon/public para NEXT_PUBLIC_SUPABASE_ANON_KEY. Puede usarse en frontend, pero no pegarla en el PDF ni en prompts publicos.
Supabase DB connection	Connection string o acceso CLI para aplicar migraciones. Guardar solo en entorno seguro.
Supabase service role	Solo si se necesita para tareas admin/server-side. Nunca en frontend. Evitar pegarla en chats.
Vercel	Cuenta/proyecto/equipo, si ya existe. Variables de entorno y dominio de preview/produccion.
WhatsApp real	Numero comercial de Victoriosa con formato internacional, por ejemplo +598XXXXXXXX.
Instagram/TikTok	Usuarios reales de redes para CTAs y growth.
Direccion/horarios	Direccion, ciudad, zona de entrega/retiro y horarios de atencion.

Dominio	Dominio deseado, por ejemplo victoriosa.com.uy o subdominio temporal.
Margen por defecto	Porcentaje inicial recomendado: 60% a 90% segun producto. Definir margen minimo aceptable.
Proveedores	Metodo real de carga: CSV exportado, links manuales, API oficial o proveedor mayorista local.
Políticas comerciales	Cambios, devoluciones, tiempos de entrega, productos bajo pedido, seña y cancelaciones.
Pagos futuros	Mercado Pago, PayPal, transferencia, efectivo o POS. No activar live en MVP sin revision.
Imagenes/logo	Logo final, colores exactos, fotos propias o banco permitido. No usar imagenes con copyright dudoso.

Regla de seguridad para claves

- No poner contraseñas, tokens, service_role, database password ni API keys en prompts largos.
- Crear .env.example con nombres de variables, pero llenar .env.local manualmente.
- En Vercel, cargar variables desde Project Settings > Environment Variables.
- El service_role solo debe usarse en funciones server-side o scripts seguros, nunca con NEXT_PUBLIC.

Prompt 00 - Director maestro de Victoriosa

Objetivo: Prompt base para usar cuando quieras que Copilot/Codex tome decisiones de arquitectura y continuidad.

Actua como sistema autonomo senior para crear y evolucionar Victoriosa, compuesto por:

- CTO de plataforma ecommerce
- Lead Full-Stack Engineer
- Supabase/RLS Security Engineer
- UI/UX Designer
- Ecommerce Product Manager
- Dropshipping Operations Manager
- AI Commerce Agent Architect
- Growth/SEO Engineer
- QA Lead
- Release Manager
- Auditor tecnico independiente

PROYECTO

Victoriosa sera una tienda propia tipo Shopify simplificado para estetica, belleza, skincare, cuidado facial, cuidado corporal y productos importados bajo modelo dropshipping/controlado.

REPOSITORIO

<https://github.com/FileLanderScanner/Victoriosa-marketplace.git>

SUPABASE

URL publica: <https://ngliugfcwydnfbpalkpb.supabase.co>
Project ref inferido: ngliugfcwydnfbpalkpb

MISION

Crear desde cero o completar el proyecto Victoriosa como plataforma comercial realista:

1. Storefront publico.
2. Catalogo de productos.
3. Servicios de estetica.
4. Evaluacion online.
5. Agenda por WhatsApp.
6. Carrito/bolsa de consulta.
7. Admin tipo Shopify simplificado.
8. Carga manual de productos.
9. Importacion CSV/JSON.
10. Proveedores mock/preparados: AliExpress, Alibaba, CJ Dropshipping, Amazon, manual.
11. Pricing, margen y scoring de rentabilidad.
12. AI Commerce Agent que sugiera productos y cree borradores, sin publicar automaticamente.
13. Supabase preparado con Auth, DB, RLS, Storage y Edge Functions si corresponde.
14. Deploy preview seguro.

REGLA CENTRAL

No preguntes que hacer despues. Trabaja en ciclos: inspeccion, priorizacion, ejecucion, validacion, documentacion, autoevaluacion y siguiente paso.

LIMITES DE SEGURIDAD

Prohibido:

- Pegar secretos en codigo.
- Usar service_role en frontend.
- Desactivar RLS.
- Activar pagos reales sin autorizacion humana.
- Hacer scraping no autorizado.
- Publicar productos automaticamente sin revision humana.
- Inventar stock, certificaciones, origen o resultados medicos.
- Usar claims tipo milagroso, cura o resultado garantizado.

Permitido:

- Crear arquitectura completa.
- Crear datos mock realistas.
- Crear importador CSV/JSON.
- Crear conectores mock.
- Crear scoring local.
- Crear productos como draft.
- Preparar integraciones futuras.
- Documentar variables y pasos.

CRITERIO DE EXITO

El ciclo es exitoso si Victoriosa queda mas cerca de una tienda real que pueda vender por WhatsApp, validar productos, revisar margenes y preparar automatizacion comercial sin comprometer seguridad.

ENTREGABLE FINAL DE CADA CICLO

Entregar reporte con:

- Rama usada
- Commits
- PR creado/actualizado
- Cambios implementados
- Checks ejecutados
- Resultado de lint/build/tests
- Riesgos
- Bloqueos humanos
- Estado de Supabase
- Estado de Vercel/preview
- Siguiendo prompt recomendado

Ejecuta el mejor ciclo posible ahora.

Prompt 01 - Preflight, repo y rama segura

Objetivo: Usalo primero en VS Code/Copilot para evitar trabajar en una carpeta equivocada o romper el repo.

Objetivo de este ciclo: preparar el repo Victoriosa de forma segura antes de tocar código.

Repo objetivo:

<https://github.com/FileLanderScanner/Victoriosa-marketplace.git>

Acciones:

1. Verificar carpeta actual con pwd y ls.
2. Si la carpeta no es repo git, clonar <https://github.com/FileLanderScanner/Victoriosa-marketplace.git>.
3. Si ya es repo git, verificar remote con git remote -v.
4. Verificar rama actual con git branch --show-current.
5. Ejecutar git status.
6. Ejecutar git fetch --all --prune si existe remote.
7. Crear rama segura: codex/victoriosa-bootstrap-marketplace.
8. No hacer force push.
9. No borrar archivos existentes sin revisar.
10. Inspeccionar package.json, src, app, docs, README si existen.
11. Determinar si el proyecto esta vacio, incompleto o ya inicializado.

Si el repo esta vacio:

- Preparar inicializacion Next.js en el siguiente ciclo.

Si el repo ya tiene código:

- No sobrescribir masivamente.
- Crear reporte de estructura actual.
- Proponer cambios incrementales.

Validaciones:

- git status limpio antes de cambios importantes.
- No secrets detectables.
- No .env real commiteado.

Entregable:

- Reporte Preflight con estado del repo, rama, estructura, riesgos y siguiente acción.

Prompt 02 - Bootstrap Next.js + arquitectura base

Objetivo: Crea la base del proyecto con estructura escalable y variables seguras.

Objetivo de este ciclo: crear la base tecnica de Victoriosa con Next.js App Router, TypeScript y Tailwind.

Contexto:

Repo: <https://github.com/FileLanderScanner/Victoriosa-marketplace.git>

Marca: Victoriosa

Modelo: tienda propia tipo Shopify simplificado + dropshipping preparado + estetica profesional.

Si la carpeta esta vacia o no tiene Next.js:

Ejecutar:

`npx create-next-app@latest . --typescript --tailwind --eslint --app --src-dir --import-alias "@/**"`

Si ya existe Next.js:

- No reinicializar.
- Adaptar la estructura existente.

Crear estructura:

```
src/app
src/app/admin
src/app/productos
src/app/servicios
src/app/agenda
src/app/evaluacion-online
src/app/sobre-victoriosa
src/app/contacto
src/app/carrito-consulta
src/components/admin
src/components/commerce
src/components/forms
src/components/layout
src/components/marketing
src/components/ui
src/data
src/lib/commerce
src/lib/suppliers
src/lib/ai
src/lib/pricing
src/lib/seo
src/types
docs
```

Crear o actualizar:

- README.md
- .env.example
- docs/ROADMAP.md
- docs/SUPPLIERS.md
- docs/AI_COMMERCE_AGENT.md
- docs/PRODUCT_IMPORT_FORMAT.md

.env.example debe incluir solo placeholders:

```
NEXT_PUBLIC_SITE_URL=http://localhost:3000
NEXT_PUBLIC_BRAND_NAME=Victoriosa
NEXT_PUBLIC_SUPABASE_URL=https://ngliugfcwydnfbpalkpb.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=replace_me
NEXT_PUBLIC_WHATSAPP_NUMBER=+598XXXXXXX
ADMIN_DEMO_MODE=true
DEFAULT_MARGIN_PERCENT=70
```

No crear .env.local con secretos reales.
No commitear .env.

Diseño:

- Elegante, femenino, premium accesible.
- Paleta rosa nude, champagne, blanco calido, beige suave, dorado sutil y texto oscuro.
- Mobile-first.

Validar:

```
npm run lint
npm run build
```

Entregable:

- Proyecto base funcionando.
- Rutas skeleton creadas.
- README inicial.
- ROADMAP inicial.
- Reporte con comandos y resultado.

Prompt 03 - Supabase foundation segura

Objetivo: Prepara Supabase sin exponer secretos y deja listo el proyecto para migraciones.

Objetivo de este ciclo: preparar Supabase foundation segura para Victoriosa.

Supabase URL:

<https://ngliugfcwydnfbpalkpb.supabase.co>

Project ref inferido:

ngliugfcwydnfbpalkpb

Importante:

No pegar claves secretas en codigo ni en comentarios.

No usar service_role en frontend.

No desactivar RLS.

No aplicar migraciones destructivas sin documentar.

Acciones:

1. Crear cliente Supabase browser/server si el proyecto ya esta listo para usarlo.

2. Usar variables:

NEXT_PUBLIC_SUPABASE_URL

NEXT_PUBLIC_SUPABASE_ANON_KEY

3. Crear lib segura:

src/lib/supabase/client.ts

src/lib/supabase/server.ts si aplica

4. Crear docs/SUPABASE_SETUP.md con:

- URL del proyecto

- variables necesarias

- como cargar anon key

- como aplicar migraciones

- advertencia service_role solo server-side

5. Preparar carpeta supabase/migrations si no existe.

6. Preparar scripts package.json si aplica:

- supabase:types

- db:migrate o documentar comando CLI

7. Crear .gitignore correcto para .env.local.

No crear tablas aun salvo que el siguiente prompt lo pida.

No inventar credenciales.

Validar:

- npm run lint

- npm run build

- revisar que no haya NEXT_PUBLIC_SUPABASE_SERVICE_ROLE

Entregable:

- Base Supabase segura preparada.

- Documentacion de configuracion.

- Lista de datos faltantes para conectar realmente.

Prompt 04 - Modelo ecommerce y migraciones

Objetivo: Define base de datos, estados de producto, proveedores, importaciones, leads y auditoria.

Objetivo de este ciclo: crear el modelo ecommerce realista en Supabase con RLS.

Tablas recomendadas:

- profiles

- products

- product_images

- suppliers

- product_import_batches

- product_import_rows

- services

- leads

- consultation_cart_events

- ai_product_recommendations

- admin_audit_events

Estados de products:

- draft

- published

- archived

source_type:

- manual

- csv

- aliexpress

- alibaba

- cj

- amazon

- other

Reglas clave:

1. Publico puede leer solo `products.status = 'published'`.
2. Publico puede leer solo `services` activos/publicados.
3. Leads se pueden insertar desde formularios publicos si se valida server-side o via endpoint seguro.
4. Admin puede gestionar productos, proveedores, servicios, leads e importaciones.
5. Importaciones crean productos como draft, nunca published automaticamente.
6. `admin_audit_events` debe registrar cambios importantes.

RLS:

- Activar RLS en todas las tablas sensibles.
- Crear funcion segura `is_admin` si existe Auth/perfiles.
- No relajar politicas para pasar tests.
- No usar `service_role` en cliente.

Crear migracion SQL en supabase/migrations con nombre timestamp claro.
Crear seed/mock opcional separado, no mezclado con migracion destructiva.

Crear types TypeScript alineados:
`src/types/index.ts`

Crear datos fallback mock si Supabase no esta configurado aun.

Validar:

- lint
- build
- si hay Supabase CLI configurado, validar migracion local/remota segun entorno.

Entregable:

- Migracion segura.
- Tipos TS.
- Docs de modelo.
- Reporte RLS y riesgos.

Prompt 05 - Storefront publico

Objetivo: Construye la tienda visible para clientas y compradores.

Objetivo de este ciclo: construir el storefront publico de Victoriosa.

Rutas publicas obligatorias:

```
/
/productos
/productos/[slug]
/servicios
/agenda
/evaluacion-online
/sobre-victoriosa
/contacto
/carrito-consulta
```

Home debe incluir:

- Hero: "Sentite cuidada, segura y victoriosa"
- CTA: Ver productos
- CTA: Reservar evaluacion
- CTA: Consultar por WhatsApp
- Servicios destacados
- Productos destacados
- Seccion: Productos seleccionados con criterio
- Seccion: Belleza + ecommerce inteligente
- Como funciona
- Testimonios mock
- CTA final

/productos:

- Filtros por categoria
- Busqueda simple
- Cards de producto
- Precio
- Estado: disponible, bajo pedido, preventa, consultar stock
- CTA WhatsApp
- Agregar a consulta


```
/productos/[slug]:
- Detalle completo
- Imagen placeholder o mock
- Beneficios
- Modo de uso
- Precauciones
- Tiempo estimado de entrega
- Proveedor visible de forma segura
- Productos relacionados

/servicios:
- Limpieza facial profunda
- Radiofrecuencia
- Criolipolisis
- Drenaje linfatico
- Tratamientos corporales
- Evaluacion estetica personalizada
- Asesoramiento de rutina facial

/evaluacion-online:
- Formulario con validacion frontend
- Aviso: "La evaluacion online no reemplaza una consulta profesional presencial. Sirve para orientar el primer contacto."
- Envio por WhatsApp

/agenda:
- Solicitud de reserva por WhatsApp
- Aclarar que el horario se confirma por respuesta humana.

Estilo:
- Elegante, limpio, premium accesible.
- Mobile excelente.
- Sin promesas medicas ni resultados garantizados.

Validar:
npm run lint
npm run build

Entregable:
- Storefront navegable.
- CTAs funcionando.
- Productos y servicios visibles.
- Reporte de rutas.
```

Prompt 06 - Admin tipo Shopify simplificado

Objetivo: Crea el panel administrativo para productos, proveedores, importaciones, leads y configuracion.

Objetivo de este ciclo: crear admin tipo Shopify simplificado para Victoriosa.

```
Rutas admin:
/admin
/admin/productos
/admin/productos/nuevo
/admin/importar
/admin/proveedores
/admin/ai-commerce
/admin/leads
/admin/configuracion

/admin:
Dashboard con metricas mock o reales si Supabase esta conectado:
- Productos publicados
- Productos en borrador
- Productos importados
- Consultas recibidas
- Servicios activos
- Margen promedio estimado
- Productos recomendados por IA
- Productos con bajo margen
- Productos pendientes de revision
```

/admin/productos:

- Tabla/lista de productos
- Filtros por estado, categoria y proveedor
- Acciones: ver, editar, publicar, despublicar, archivar, marcar draft
- Botones: crear producto, importar productos, revisar sugerencias IA

/admin/productos/nuevo:

Formulario:

- nombre
- slug
- categoria
- descripcion corta/larga
- beneficios
- modo de uso
- precauciones
- precio costo
- envio
- otros costos
- margen deseado
- precio sugerido
- precio venta
- proveedor
- url proveedor
- imagen URL
- estado draft/published/archived
- stock manual o bajo pedido

/admin/importar:

- Importar CSV
- Pegar JSON
- Vista previa
- Validacion
- Crear borradores
- Advertencia revision humana

/admin/proveedores:

- Gestion basica de proveedores manual/mock

/admin/ai-commerce:

- Productos sugeridos
- Scores
- Motivo de recomendacion
- Accion sugerida
- Crear draft, descartar, revisar proveedor, ajustar precio

/admin/leads:

- Leads mock/reales
- Estado: nuevo, contactado, convertido, descartado
- Abrir WhatsApp

/admin/configuracion:

- WhatsApp
- Marca
- Instagram
- Direccion
- Horarios
- Margen por defecto
- Moneda
- Pais objetivo
- Publicacion automatica: desactivada por defecto

Seguridad:

- Si no hay Auth real, mostrar claramente ADMIN_DEMO_MODE.
- Preparar proteccion futura.
- No depender de seguridad solo frontend para produccion.

Validar:

```
npm run lint
npm run build
```

Entregable:

- Admin navegable.
- Formularios base.
- Reporte de limitaciones y proxima fase Auth/RLS.

Prompt 07 - Importador CSV/JSON y proveedores mock

Objetivo: Implementa carga de productos importados sin scraping ni APIs privadas.

Objetivo de este ciclo: implementar importador de productos CSV/JSON y conectores mock de proveedores.

Crear:

```
src/lib/commerce/importProducts.ts
src/lib/suppliers/types.ts
src/lib/suppliers/supplierConnector.interface.ts
src/lib/suppliers/mockAliExpressConnector.ts
src/lib/suppliers/mockAlibabaConnector.ts
src/lib/suppliers/mockCJConnector.ts
src/lib/suppliers/mockAmazonConnector.ts
```

```
docs/PRODUCT_IMPORT_FORMAT.md
docs/SUPPLIERS.md
```

Importador debe:

- Parsear CSV simple.
- Parsear JSON pegado.
- Validar campos requeridos.
- Normalizar nombres.
- Generar slug.
- Calcular costo total.
- Calcular precio sugerido.
- Calcular margen.
- Asignar status = draft.
- Devolver errores por fila.
- Mostrar preview antes de confirmar.

Campos CSV minimos:

title, category, shortDescription, costPrice, shippingCost, targetMarginPercent, supplierName, supplierUrl, imageUrl, sourceType

Conectores mock:

- No llamar APIs reales.
- No hacer scraping.
- Simular busqueda de productos.
- Normalizar producto externo a ProductDraft.
- Marcar origen aliexpress/alibaba/cj/amazon/manual.
- Documentar que produccion requiere API autorizada, CSV exportado o carga manual.

Regla critica:

Todo producto importado queda como draft. Nunca publicar automaticamente.

UI:

/admin/importar debe permitir:

- Pegar CSV
- Pegar JSON
- Ver preview
- Ver errores
- Crear borradores simulados o reales si Supabase esta listo

Validar:

```
npm run lint
npm run build
```

Entregable:

- Importador funcional frontend/local.
- Conectores mock.
- Docs con formato CSV y riesgos de dropshipping.

Prompt 08 - Pricing, margen y scoring rentable

Objetivo: Agrega calculo de precio, margen y seleccion preliminar de productos rentables.

Objetivo de este ciclo: implementar pricing, margen y scoring de rentabilidad para Victoriosa.

Crear:

```
src/lib/pricing/calculatePrice.ts
```

```
src/lib/ai/productScoring.ts
src/components/commerce/ProfitBadge.tsx
src/components/admin/AIRecommendationCard.tsx
```

Funciones pricing:

- calculateTotalCost(costPrice, shippingCost, extraCosts)
- calculateSuggestedPrice(totalCost, targetMarginPercent)
- calculateProfit(salePrice, totalCost)
- calculateMarginPercent(salePrice, totalCost)
- roundToCommercialPrice(price, currency)
- calculateProfitScore(product)

Regla inicial:

precio sugerido = costo total * (1 + margen deseado / 100)

Redondeo comercial UYU:

- Usar terminaciones tipo 490, 590, 690, 790, 890, 990 cuando tenga sentido.
- Evitar precios raros como 743.28.

Scoring local mock:

- profitScore: margen alto = mejor.
- demandScore: skincare, cuidado facial, proteccion solar y limpieza = mayor demanda estimada.
- riskScore: envio largo, proveedor baja confiabilidad o precio alto = mayor riesgo.
- finalScore: combinacion ponderada.
- recommendation: texto claro para admin.

No afirmar demanda real si no hay datos reales.

Usar expresiones:

- demanda estimada
- score interno
- recomendacion preliminar
- revisar proveedor

Mostrar en admin:

- Alto margen
- Margen bajo
- Riesgo proveedor
- Revisar precio
- Apto para borrador

Validar:

```
npm run lint
npm run build
```

Entregable:

- Helpers testeables.
- Badges visibles.
- Admin con scoring comprensible.

Prompt 09 - AI Commerce Agent

Objetivo: Crea el asistente comercial de IA en modo seguro y preparado para automatizacion futura.

Objetivo de este ciclo: crear el modulo AI Commerce Agent de Victoriosa en modo seguro/mock.

Nombre del agente:

Victoriosa AI Commerce Agent

Responsabilidades futuras:

1. Buscar productos rentables en proveedores autorizados.
2. Comparar costo, envio y margen.
3. Proponer precio de venta.
4. Generar titulo comercial.
5. Generar descripcion SEO.
6. Crear producto en borrador.
7. Sugerir bundles.
8. Detectar productos con bajo margen.
9. Detectar productos de alto riesgo.
10. Sugerir campañas para WhatsApp/Instagram.

Implementacion MVP:

- No usar API IA real.

- Usar funciones locales mock.
- Crear sugerencias desde products/suppliers mock.
- Permitir crear draft desde sugerencia.
- Mostrar motivo de sugerencia.

Crear:

```
src/lib/ai/generateProductCopy.ts
src/lib/ai/generateSeo.ts
src/lib/ai/generateBundles.ts
src/lib/ai/productRecommendations.ts
src/data/aiRecommendations.ts
docs/AI_COMMERCE_AGENT.md
```

Reglas de seguridad:

- No publica automaticamente.
- No compra automaticamente.
- No usa scraping.
- No inventa stock.
- No inventa certificaciones.
- No usa resenas falsas.
- No copia marcas registradas.
- No promete resultados medicos.
- Todo producto sugerido entra como draft.
- Admin humano aprueba.

UI /admin/ai-commerce:

- Lista de sugerencias.
- Score final.
- Margen esperado.
- Riesgo.
- Motivo.
- Accion: crear draft, descartar, revisar proveedor, ajustar precio.

Validar:

```
npm run lint
npm run build
```

Entregable:

- Panel AI Commerce mock funcional.
- Docs de futura automatizacion.
- Reporte de limites actuales.

Prompt 10 - WhatsApp, leads y carrito de consulta

Objetivo: Convierte la tienda en un embudo de ventas simple sin pagos reales.

Objetivo de este ciclo: implementar whatsapp comercial, leads y carrito/bolsa de consulta.

Crear:

```
src/lib/whatsapp.ts
src/components/commerce/CartProvider.tsx
src/app/carrito-consulta/page.tsx
src/components/forms/AppointmentForm.tsx
src/components/forms/OnlineEvaluationForm.tsx
src/components/forms/ContactForm.tsx
```

whatsapp.ts debe incluir:

- VICTORIOSA_WHATSAPP placeholder desde NEXT_PUBLIC_WHATSAPP_NUMBER
- generateWhatsAppUrl
- generateProductInquiryMessage
- generateCartInquiryMessage
- generateServiceInquiryMessage
- generateAppointmentMessage
- generateEvaluationMessage

Carrito de consulta:

- Agregar producto.
- Quitar producto.
- Cambiar cantidad.
- Persistir en localStorage.
- Generar mensaje WhatsApp con lista.
- No cobrar.
- No prometer stock.

Leads:

- Si Supabase esta listo, preparar insercion segura.
- Si no, usar mock/local y documentar.
- Estados: nuevo, contactado, convertido, descartado.

Formularios:

Agenda:

- nombre, telefono, servicio, fecha, horario, comentario.

Evaluacion:

- nombre, edad, objetivo, tipo de piel, zona, sensibilidad, embarazo/lactancia, enfermedades, medicacion, productos actuales, disponibilidad.

Contacto:

- nombre, telefono, mensaje.

Avisos:

- Horario sujeto a confirmacion por WhatsApp.
- Producto sujeto a disponibilidad.
- Evaluacion online no reemplaza consulta presencial.

Validar:

npm run lint
npm run build

Entregable:

- WhatsApp funcionando con mensajes prellenados.
- Carrito consulta funcional.
- Formularios validados.

Prompt 11 - Seguridad, RLS y QA

Objetivo: Audita el proyecto antes de publicar previews o avanzar a pagos.

Objetivo de este ciclo: hacer auditoria de seguridad, RLS, calidad y QA de Victoriosa.

Auditar:

1. No hay secretos en repo.
2. No hay .env.local commiteado.
3. No hay service_role en frontend.
4. No hay APIs de proveedores reales sin autorizacion.
5. No hay scraping.
6. No hay pagos live.
7. No hay claims medicos peligrosos.
8. Productos importados quedan draft.
9. Publico solo ve productos published.
10. Admin demo esta claramente marcado si no hay Auth real.

Si Supabase esta implementado:

- Revisar RLS tabla por tabla.
- Crear smoke tests de RLS si es viable.
- Confirmar lectura publica limitada.
- Confirmar escritura admin-only donde corresponda.
- Confirmar leads insertables sin exponer datos de otros leads.

QA UI:

- Rutas publicas cargan.
- Rutas admin cargan.
- Mobile responsive.
- Formularios validan.
- WhatsApp abre con mensaje correcto.
- Importador muestra errores por fila.
- Build sin errores.

Comandos:

npm run lint
npm run build
npm test si existe
npm run typecheck si existe

Si scripts no existen:

- No inventar resultado.
- Crear script si corresponde o documentar ausencia.

Entregable:

- Security/QA Report.
- Lista de riesgos.
- Fixes aplicados.
- Bloqueos humanos.

Prompt 12 - Deploy preview Vercel y variables

Objetivo: Prepara despliegue seguro sin tocar produccion.

Objetivo de este ciclo: preparar deploy preview seguro en Vercel.

Repo:

<https://github.com/FileLanderScanner/Victoriosa-marketplace.git>

Supabase URL publica:

<https://ngliugfcwydnfbpalkpb.supabase.co>

No hacer deploy production salvo autorizacion humana explicita.

No activar pagos live.

No exponer secrets.

Acciones:

1. Verificar build local.
2. Crear/actualizar README con variables necesarias.
3. Crear checklist de Vercel:
 - NEXT_PUBLIC_SITE_URL
 - NEXT_PUBLIC_BRAND_NAME=Victoriosa
 - NEXT_PUBLIC_SUPABASE_URL=<https://ngliugfcwydnfbpalkpb.supabase.co>
 - NEXT_PUBLIC_SUPABASE_ANON_KEY
 - NEXT_PUBLIC_WHATSAPP_NUMBER
 - ADMIN_DEMO_MODE
 - DEFAULT_MARGIN_PERCENT
4. Si Vercel CLI esta autenticado y autorizado, crear preview deploy.
5. Si no esta autenticado, dejar comandos exactos.
6. Verificar rutas de preview:
 - /
 - /productos
 - /servicios
 - /agenda
 - /evaluacion-online
 - /admin
 - /admin/importar
 - /admin/ai-commerce
7. Documentar resultado.

Protecciones:

- Produccion: NO-GO hasta que haya Auth admin, RLS validado, politicas comerciales y revision humana.
- Preview: OK para revision controlada.

Entregable:

- Preview URL si se pudo crear.
- Variables requeridas.
- Checklist Vercel.
- Estado GO/NO-GO produccion.

Prompt 13 - Growth, SEO y venta piloto

Objetivo: Prepara la plataforma para validar ventas sin comprar automatizaciones caras.

Objetivo de este ciclo: preparar Victoriosa para venta piloto, SEO local y primeras campañas.

Acciones:

1. Mejorar textos de home, productos y servicios con enfoque conversion.
2. Crear metadatos SEO por pagina.
3. Crear FAQ comercial:
 - Como comprar

- Como consultar stock
- Tiempos de entrega
- Productos bajo pedido
- Cambios/devoluciones
- Agenda estetica
- Evaluacion online
- 4. Crear bundles sugeridos:
 - Kit limpieza facial
 - Kit hidratacion
 - Rutina skincare inicial
 - Kit cuidado corporal
- 5. Crear landing sections para:
 - Productos para rutina facial
 - Productos bajo pedido
 - Evaluacion estetica online
- 6. Crear docs/GROWTH_PLAN.md con:
 - Primeras 20 publicaciones Instagram/TikTok
 - Guiones cortos para reels
 - Mensajes WhatsApp de venta
 - Campana de lanzamiento
 - Oferta inicial sin promesas falsas

Reglas:

- No inventar resultados garantizados.
- No usar resenas falsas como si fueran reales.
- Testimonios mock deben ser claramente de ejemplo si se documenta internamente.
- No vender productos regulados o falsificados.

Validar:

```
npm run lint
npm run build
```

Entregable:

- Sitio mas vendible.
- Plan Growth.
- CTAs claros.
- Reporte de mejoras.

Prompt 14 - Reporte final y continuidad autonoma

Objetivo: Cierra cada ciclo con evidencia, riesgos y siguiente accion concreta.

Objetivo de este ciclo: cerrar el trabajo con reporte director y dejar siguiente ciclo listo.

Generar un Victoriosa Director Report con:

1. Modo ejecutado.
2. Rama usada.
3. Commits creados.
4. PR creado o actualizado.
5. Estado del PR.
6. Checks ejecutados y resultado.
7. Rutas publicas implementadas.
8. Rutas admin implementadas.
9. Estado de Supabase.
10. Estado de Vercel/preview.
11. Estado de seguridad:
 - secrets
 - RLS
 - service_role
 - pagos
 - proveedores
12. Estado ecommerce:
 - productos
 - importador
 - pricing
 - scoring
 - carrito consulta
13. Estado AI Commerce:
 - mock
 - recomendaciones
 - limites
14. Riesgos detectados.
15. Bloqueos humanos.
16. Produccion: GO / NO-GO.
17. Autoevaluacion:

- Avanzo hacia MVP real?
- Redujo riesgo tecnico?
- Mantiene seguridad?
- Mejora capacidad de vender?
- Prepara automatizacion rentable?

18. Proximo paso recomendado.

19. Nuevo prompt sugerido para continuar.

Si hay cambios sin commit:

- Mostrar git status.
- Crear commit si corresponde.
- No force push.

Si hay PR posible:

- Crear PR con descripcion clara.
- No mergear si requiere review humana.

Entregable final:

Reporte claro, corto y accionable.

Prompt 15 - Mensaje corto para Copilot

Objetivo: Versión breve para pegar cuando quieras iniciar rápido sin todo el documento.

Ejecuta los prompts maestros de Victoriosa en orden, empezando por el Prompt 01.
Usa el repositorio configurado, crea rama segura, no pegues secretos, no uses service_role en frontend, no hagas scraping, no actives pagos reales y no publiques productos importados automaticamente.

Prioridad del primer ciclo:

1. Preflight repo.
2. Bootstrap Next.js si hace falta.
3. Estructura base.
4. Storefront publico.
5. Admin basico.
6. Importador CSV/JSON.
7. Pricing y scoring.
8. WhatsApp comercial.
9. Lint y build.
10. Reporte final.

Checklist humano antes de conectar produccion

- Confirmar WhatsApp real y probar mensajes desde mobile.
- Cargar NEXT_PUBLIC_SUPABASE_ANON_KEY en .env.local y Vercel, no en codigo.
- Confirmar RLS con usuarios admin/publicos antes de guardar leads reales.
- Definir politicas de cambios, devoluciones y tiempos de entrega.
- Validar proveedores y evitar productos regulados, falsificados o con claims medicos.
- Usar pagos live solo cuando haya ordenes, terminos, soporte y revision legal/comercial.
- Mantener publicacion automatica desactivada hasta tener datos reales de ventas y reclamos.

Fin del documento - Victoriosa Prompts Maestros